Architecture, Setup & Maintenance Report

Phishing URL Detection — Deployment Report (Week 7)

INTERN DETAILS

Intern Name	Rushik Rajendra Kokate
Intern ID	#37018
Program	Code B - Data Science Integrated Internship
Organization	ITVedant
Date	March 2026
Week	7 of 8
GitHub	Link

1. Architecture Overview

1.1 High-Level System Design

```
┌──────────────────────────────────────────────────────────────────────┐
│                          CLIENT LAYER                                │
│                                                                      │
│   Browser / cURL / Security Tool / SIEM / Browser Extension          │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              │  HTTPS Request
                              │  POST /predict  { feature_json }
                              │  GET  /health
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                       RENDER WEB SERVICE                             │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │                      GUNICORN (WSGI)                         │   │
│   │                2 workers  |  port $PORT                      │   │
│   │                                                              │   │
│   │   ┌──────────────────────────────────────────────────────┐   │   │
│   │   │                    FLASK (app.py)                    │   │   │
│   │   │                                                      │   │   │
│   │   │  Routes:                                             │   │   │
│   │   │    GET  /              → API info                    │   │   │
│   │   │    GET  /health        → health check                │   │   │
│   │   │    GET  /model/info    → metadata + metrics          │   │   │
│   │   │    GET  /features      → feature importances         │   │   │
│   │   │    POST /predict       → single prediction           │   │   │
│   │   │    POST /predict/batch → batch (≤100 records)        │   │   │
```

```
│  │  │                                                │  │   │
│  │  │  Request flow:                                 │  │   │
│  │  │    1. Parse JSON body                          │  │   │
│  │  │    2. Validate 99 features present             │  │   │
│  │  │    3. Build pandas DataFrame                   │  │   │
│  │  │    4. model.predict_proba(X)                   │  │   │
│  │  │    5. Compute local feature attributions       │  │   │
│  │  │    6. Return JSON prediction + explanations    │  │   │
│  │  └────────────────────────────────────────────────┘  │   │
│  │                                                      │   │
│  │  ┌────────────────────┐                              │   │
│  │  │   model.pkl        │  ← Loaded once at startup    │   │
│  │  │   (in-memory cache) │    Stays resident in RAM    │   │
│  │  └────────────────────┘                              │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                              │
                              │  JSON Response
                              ▼
          {
            "prediction": "phishing",
            "phishing_probability": 0.9800,
            "confidence": 0.9800,
            "top_contributing_features": { ... }
          }
```

1.2 Request-Response Lifecycle

```
1. Client sends POST /predict with 99-feature JSON object
        ↓
2. Gunicorn worker receives request, routes to Flask
        ↓
3. Flask parses JSON → validates all 99 features present
        ↓
4. Features assembled into pandas DataFrame (1 × 99)
        ↓
5. model.predict_proba(X) returns [P(legit), P(phishing)]
        ↓
6. P(phishing) compared to threshold (default: 0.5)
        ↓
7. Local feature attribution computed:
   attribution_i = feature_importance_i × |feature_value_i|
   normalised to sum to 1
        ↓
8. Top 10 contributing features extracted
        ↓
9. JSON response returned:
   { prediction, phishing_probability, confidence,
     top_contributing_features }
        ↓
10. Gunicorn returns HTTP 200 to client
```

1.3 Technology Stack

Layer	Technology	Version	Purpose
ML Model	scikit-learn RandomForest	1.5.2	Core predictor
Web Framework	Flask	3.0.3	REST API
WSGI Server	Gunicorn	22.0.0	Production HTTP server
Serialisation	pickle	stdlib	Model persistence
Data processing	pandas + numpy	2.2.3 / 1.26.4	Feature handling
Explainability	SHAP (+ sklearn MDI)	0.45.1	Feature attributions
Platform	Render	—	Cloud hosting
Language	Python	3.11	Runtime

---

2. Setup Instructions

2.1 Local Development Environment

Requirements

- Python 3.10 or 3.11 (3.12 compatible)
- pip 23+
- Git
- At least 1 GB free RAM (model loading)

Step 1 — Clone and install

```
git clone https://github.com/Kokate-Rushik/phishing-detector-api.git
cd phishing-detector-api
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Step 2 — Generate model

```
# Requires train_refined.csv and test_refined.csv (Week 4 outputs)
cp ../Week4/train_refined.csv .
cp ../Week4/test_refined.csv  .
python generate_model.py
# Outputs: model.pkl, model_meta.json, shap_analysis.csv, local_explanations.json
```

Step 3 — Run locally

```
# Development (single-threaded Flask dev server)
python app.py

# Production-equivalent (Gunicorn, mirrors Render)
gunicorn app:app --workers 2 --bind 0.0.0.0:5000 --timeout 120
```

Step 4 — Test locally

```
# Health check
curl http://localhost:5000/health

# Model info
curl http://localhost:5000/model/info

# Single prediction (requires full 99-feature JSON)
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d @tests/sample_phishing.json

# Batch prediction
curl -X POST http://localhost:5000/predict/batch \
  -H "Content-Type: application/json" \
  -d '{"records": [{...}, {...}]}'
```

2.2 Project Directory Structure

```
phishing-detector-api/            ← Week7 folder
├── model/
│   ├── local_explanations.json   ← Sample prediction explanations
│   ├── model_meta.json           ← Model metadata reference
│   └── model.pkl                 ← Serialised Random Forest (the "Brain")
├── test/
│   ├── batch-test.json           ← Batch prediction test payload
│   ├── legitmate.json            ← Legitimate URL test payload
│   └── phishing.json             ← Phishing URL test payload
├── app.py                        ← Flask application
├── generate_model.py             ← Model training + SHAP analysis script
├── Documentation.md              ← Interpretability + deployment docs
├── render.yaml                   ← Render deployment configuration
├── Report.md                     ← Final summary report
├── requirements.txt              ← Python dependencies
└── shap_analysis.csv             ← Global feature importances (SHAP values)
```

2.3 Configuration Files

`render.yaml`

```yaml
services:
  - type: web
    name: phishing-detector-api
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn app:app --workers 2 --bind 0.0.0.0:$PORT --timeout 120
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.0
      - key: MODEL_PATH
        value: model/model.pkl
      - key: PREDICTION_THRESHOLD
        value: "0.5"
      - key: FLASK_DEBUG
        value: "false"
```

`.env` (local development only — never commit)

```
MODEL_PATH=model.pkl
PREDICTION_THRESHOLD=0.5
FLASK_DEBUG=true
PORT=5000
```

2.4 Render Deployment (Step-by-Step)

1. Push repository to GitHub (include `model.pkl`)
2. Log into https://render.com
3. Click **New +** → **Web Service**
4. Select your GitHub repository
5. Set:
   - **Name:** `phishing-detector-api`
   - **Region:** Oregon (US West) or nearest region
   - **Branch:** `main`
   - **Runtime:** Python 3
   - **Build Command:** `pip install -r requirements.txt`
   - **Start Command:** `gunicorn app:app --workers 2 --bind 0.0.0.0:$PORT --timeout 120`
6. Under **Environment**, add:
   - `PYTHON_VERSION` = `3.11.0`
   - `MODEL_PATH` = `model.pkl`
   - `PREDICTION_THRESHOLD` = `0.5`
7. Click **Create Web Service**
8. Monitor build logs (~3–5 minutes)
9. Service URL: `https://phishing-detector-api.onrender.com`

Note on free tier: Render free tier services spin down after 15 minutes of inactivity. The first request after spin-down takes ~20–30 seconds (cold start). Use the Starter plan ($7/month) for always-on availability.

---

3. Monitoring and Maintenance Plan

3.1 Performance Monitoring

Application-Level Metrics

Track these metrics using Render's built-in logs or by forwarding to a monitoring service (e.g., Datadog, New Relic, UptimeRobot):

Metric	Target	Alert Threshold
API response time (p95)	< 200ms	> 500ms
Error rate (5xx)	< 0.1%	> 1%
Requests per minute	Baseline ±50%	3× baseline
Memory usage	< 400 MB	> 480 MB
Model load time	< 5s (startup)	> 10s

Model-Level Metrics

Monitor prediction distribution in production to detect model drift:

```
# Add to app.py — log prediction stats periodically
import collections
prediction_log = collections.deque(maxlen=1000)

# In /predict route, after prediction:
prediction_log.append({
    "timestamp": datetime.now().isoformat(),
    "prediction": result["prediction"],
    "probability": result["phishing_probability"]
})

# Add /metrics endpoint to expose aggregate stats
@app.route("/metrics")
def metrics():
    if not prediction_log:
        return jsonify({"message": "No predictions yet"})
    probs = [p["probability"] for p in prediction_log]
    phish_rate = sum(1 for p in prediction_log if p["prediction"]=="phishing") /
len(prediction_log)
    return jsonify({
        "n_predictions": len(prediction_log),
        "phishing_rate": round(phish_rate, 4),
        "avg_probability": round(sum(probs)/len(probs), 4),
        "high_confidence_rate": round(sum(1 for p in probs if p>0.9 or
p<0.1)/len(probs), 4)
    })
```

Key Drift Indicators

Signal	Meaning	Action
Phishing rate drops below 10%	Model classifying most URLs as legitimate	Check input data pipeline; possible feature drift
Average probability clusters around 0.5	Model is uncertain — boundary shift	Collect new labeled data; retrain
`google_index` always = 0 in inputs	Data pipeline issue	Investigate feature extraction
Response time increases	Memory pressure or model load	Scale up instance; check worker count

3.2 Data Drift Detection

Phishing techniques evolve monthly. The following strategy detects when the model needs retraining:

Monthly drift check:

```
# Compare incoming feature distributions to training distribution
from scipy.stats import ks_2samp

# Store training feature stats in model_meta.json (add during generate_model.py)
# Each month, run KS test on production samples vs training distribution
for feature in TOP_10_FEATURES:
    stat, pval = ks_2samp(training_samples[feature], production_samples[feature])
    if pval < 0.05:
        print(f"DRIFT DETECTED: {feature} (p={pval:.4f})")
```

Retraining triggers:

- Any top-10 feature shows significant distribution shift (KS test $p < 0.05$)
- Production phishing rate deviates >15% from expected baseline
- New phishing campaign patterns reported by threat intelligence feeds
- Model F1 on newly labeled production samples drops below 0.94

3.3 Model Update Procedure

When retraining is needed:

```
# Step 1: Collect new labeled data
# Append new samples to dataset, re-run Week 3–5 pipeline

# Step 2: Retrain
python generate_model.py   # produces new model.pkl

# Step 3: Validate
# Ensure new model.pkl achieves F1 >= 0.95 before deploying
```

```
# Step 4: Blue-green deployment on Render
# Push new model.pkl to a staging branch → deploy to staging service
# Run smoke tests → merge to main → auto-deploy production

# Step 5: Version the old model
cp model.pkl model_v1_$(date +%Y%m%d).pkl
git tag v1.$(date +%Y%m%d)
```

3.4 Scaling Strategy

Traffic Level	Action	Cost
< 100 req/day	Free tier	$0
100–1,000 req/day	Starter plan (1 worker)	$7/month
1,000–10,000 req/day	Standard (2–4 workers)	$25/month
> 10,000 req/day	Load balancer + multiple instances	$50+/month

For high-traffic scenarios, consider caching predictions for repeated feature vectors (phishing URLs often share patterns) using Redis.

3.5 Security Hardening (Production)

Concern	Mitigation
Unauthorised API access	Add API key header validation
model.pkl tampering	Checksum validation on startup
Large payload DoS	Limit request body to 1 MB in Nginx/Gunicorn
Feature injection	Strict type validation on all 99 input features
Model extraction	Rate limit `/predict` to 100 req/min per IP

Add API key middleware:

```
API_KEY = os.environ.get("API_KEY", "")

@app.before_request
def check_api_key():
    if API_KEY and request.path not in ["/", "/health"]:
        key = request.headers.get("X-API-Key", "")
        if key != API_KEY:
            return jsonify({"error": "Unauthorized"}), 401
```

---

4. Summary

Deliverable	Status	Location
`generate_model.py`	Complete	Model training + SHAP analysis
`app.py`	Complete	Flask REST API, Render-ready
`requirements.txt`	Complete	All dependencies pinned
`Documentation.md`	Complete	Interpretability + deployment steps
`Report.md`	Complete	This document
`model.pkl`	Generated	Serialised RF (200 trees, F1=0.9621)
`shap_analysis.csv`	Generated	99-feature global importances
Render deployment	Ready	Push to GitHub → connect Render

Endpoint URL structure:

```
https://phishing-detector-api-xf0o.onrender.com/predict        ← Single
prediction
https://phishing-detector-api-xf0o.onrender.com/predict/batch  ← Batch prediction
https://phishing-detector-api-xf0o.onrender.com/health         ← Health check
https://phishing-detector-api-xf0o.onrender.com/model/info     ← Model metadata
https://phishing-detector-api-xf0o.onrender.com/features       ← Feature
importances
```

Report prepared for Week 7 — Deployment & Interpretability Assignment
Stack: Flask + Gunicorn on Render | Model: Random Forest (F1=0.9621, AUC=0.9937)